The Journal of Open Source Software

MC^{++} and PyMC^{++}: construction, manipulation and bounding of factorable functions

Benoit Chachuat [1¶], **Kai Ruth** [1,2], **and Marco Sandrin** [1]

1 Sargent Centre for Process Systems Engineering, Department of Chemical Engineering, Imperial College London, United Kingdom **2** Department of Chemistry and Applied Biosciences, ETH Zurich, Switzerland ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 04 November 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Some questions about a mathematical model cannot be settled by merely evaluating it, however many times: what values an output can take over a whole set of inputs, whether a constraint holds throughout a prescribed domain, or within what bounds the global optimum of a nonconvex problem provably lies. Such questions call for enclosures and convex relaxations, obtained by propagating sets rather than numbers through the model's expressions.

MC^{++} is a C++ toolkit that does this for factorable functions, the class obtained by composing arithmetic operations with standard intrinsic functions, which covers most algebraic models in science and engineering. A function is represented once, as a directed acyclic graph (DAG) of its operations, and that graph is then evaluated in whichever arithmetic a task needs: real numbers, derivatives, interval enclosures, convex and concave relaxations, polynomial models with remainder bounds, and others. PyMC^{++} (module `pymcpp`) brings the toolkit to Python as prebuilt wheels on PyPI.

Statement of need

Deterministic global optimization, reachability analysis of dynamic systems and rigorous model validation all rest on the same primitive: given a factorable function and a host set for its variables, produce a guaranteed enclosure of its range, or a convex relaxation a solver can minimize (Chachuat et al., 2015). Researchers in these fields, and the developers of the solvers and modeling tools that serve them, have so far had to choose between implementing an arithmetic from scratch and using one locked inside a solver. What has been missing is a library that offers several such arithmetics, composable and extensible, over one shared expression graph, and is reachable from Python, where much of the modeling work that needs rigorous bounds is done.

State of the field

Expression-graph and automatic-differentiation tools — CasADi (Andersson et al., 2019), CppAD (Bell, 2026), ADOL-C (Griewank et al., 1996), the modeling languages Pyomo (Bynum et al., 2021; Hart et al., 2011), JuMP (Lubin et al., 2023), AMPL (Fourer et al., 1990) and GAMS (Bussieck & Meeraus, 2004), and the tracing autodiff of PyTorch (Paszke et al., 2019) and JAX (Bradbury et al., 2018) — answer what $f(x)$ and $\nabla f(x)$ are *here*. What values f can take over a set, or what convex function underestimates it there, is at best handled by an extension.

Rigorous-bounding libraries are each specialized to one arithmetic: intervals in filib++ (Lerch

DRAFT

et al., 2006), PROFIL/BIAS (Knüppel, 1994), Boost.Interval (Brönnimann et al., 2006) and IBEX (Chabert & Jaulin, 2009); Taylor models in COSY Infinity (Makino & Berz, 2006), which is not openly licensed, TaylorModels.jl (Benet, Sanders, et al., 2026; Benet & Sanders, 2019), and reachability tools such as Flow* (Chen et al., 2013) and CORA (Althoff, 2015). Chebyshev model arithmetic has, to our knowledge, no open alternative. Global optimization solvers — BARON (Tawarmalani & Sahinidis, 2005), ANTIGONE (Misener & Floudas, 2014), SCIP (Hojny et al., 2025), Couenne (Belotti et al., 2009), Alpine (Nagarajan et al., 2019), Gurobi (Gurobi Optimization, LLC, 2026), EAGO (Wilhelm & Stuber, 2022) and MAiNGO (Bongartz et al., 2018) — relax factorable functions, but inside their own search and not addressable from outside. The nearest reusable alternative, the Julia stack around EAGO.jl, McCormick.jl (Wilhelm & Stuber, 2022) and IntervalArithmetic.jl (Benet, Hénot, et al., 2026), offers fewer arithmetics as separate packages and no Python interface. For Python users, the only prior route has been Pyomo's separately compiled `contrib.mcpp` extension to an earlier MC^{++} release.

Contributing these arithmetics to an existing package was not realistic. Several graph-building tools admit user-defined value types, but none was designed to host a collection of interchangeable and composable arithmetics, with one serving as the bounding type of another — which is why Pyomo delegates its bounding to MC^{++}.

Software design

Expressions are built by writing ordinary arithmetic on the variables of a DAG, and identical subexpressions are shared as they are created. For instance, the DAG of $f(x,y) = e^{xy} - 2x^2y + 3$ and $g(x,y) = \sqrt{x^2y + 4} + e^{xy}$ shares xy, e^{xy} and x^2. A *subgraph*, the operation list for chosen dependents in topological order, is a first-class object, so repeated evaluation reuses one precomputed sequence. A single routine sweeps it, with a vectorized, multithreaded form for many points and a reverse form that contracts variable ranges against bounds on the dependents.

What that routine computes is decided by the type of the values it is given. Floats evaluate the function; intervals enclose its range over the host set (Moore et al., 2009); and the relaxation types return convex and concave estimators (McCormick, 1976) with their subgradients (Mitsos et al., 2009), polynomial models with remainder bounds (Neumaier, 2003; Rajyaguru et al., 2017), or separable superposition estimators (Zha et al., 2026). Spectral bounds on Hessians (Mönnigmann, 2011), the linear cuts of a polyhedral relaxation (Smith & Pantelides, 1999; Tawarmalani & Sahinidis, 2005), and ellipsoidal enclosures (Villanueva et al., 2015) propagate the same way. Derivatives are generated symbolically, in forward or reverse accumulation (Griewank & Walther, 2008), as sparse DAG nodes that can be differentiated again or evaluated in any of these arithmetics. A DAG can also be decomposed into polynomial and transcendental subexpressions, the reformulation step that precedes relaxation in global optimization.

MC^{++} is header-only. The hinge is a single traits class, `mc::Op<T>`: any type specializing it can be propagated through a DAG, used on its own, or serve as the bounding type of another — for instance, Taylor and Chebyshev models propagating their remainder in McCormick arithmetic (Bompadre et al., 2013; Sahlodin & Chachuat, 2011), or superposition relaxations parameterizing their univariate summands as piecewise-constant or piecewise-linear functions (Zha et al., 2026). The interval type beneath them all is interchangeable — MC^{++}'s own, Boost, PROFIL/BIAS or filib++ — and directed rounding, where the backend supports it, makes the interval enclosures verified; the other enclosures remain subject to rounding errors. PyMC^{++}, built with pybind11 (Jakob et al., 2017), binds a fixed selection of these instantiations, while the full template space is open to C++ users.

The DAG is extensible through *external operations*, user-defined nodes carrying their own evaluation and differentiation rules. One such operation encapsulates opaque external code, such as a Python function or a proprietary routine; others support affine and polynomial

DRAFT

subexpressions, multilayer perceptrons imported from PyTorch, and nested DAGs. The companion library CRONOS (https://github.com/omega-icl/cronos) uses them to embed solvers for algebraic, differential and partial differential equations as single DAG nodes, while CANON (https://github.com/omega-icl/canon, optimization) and MAGNUS (https://github.com/omega-icl/magnus, uncertainty analysis) build on the same graphs.

MC^{++} is released under the Eclipse Public License 2.0, with the GNU GPL v2 or later as a secondary license.

Example

The Python listing below builds the DAG of $f(x, y) = e^{xy} - 2x^2y + 3$; then, the same `eval` call returns a value or an enclosure, depending only on the type of its inputs. Figure 1 plots the McCormick, Chebyshev and superposition enclosures over the host set $[-1, 1] \times [0, 2]$.

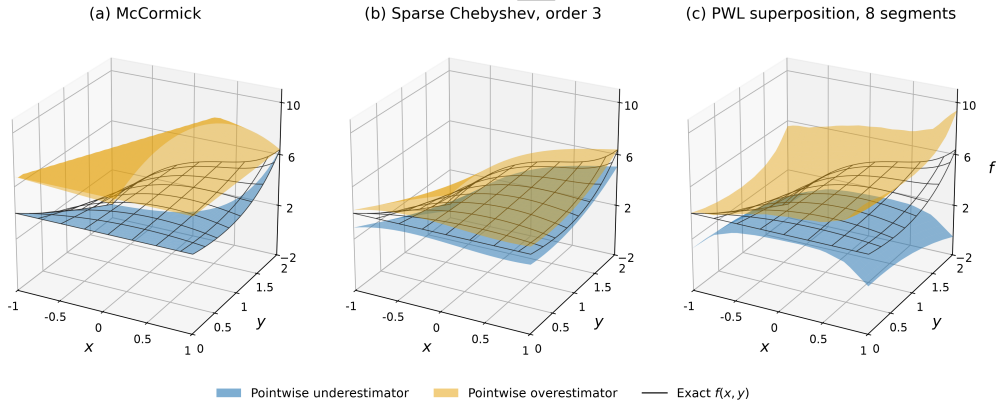


Figure 1: Enclosures of f on $[-1, 1] \times [0, 2]$: (a) McCormick relaxations, (b) third-order sparse Chebyshev model, (c) piecewise-linear superposition estimators with eight segments per variable. The exact function is the black wireframe; under- and overestimators are in blue and orange.

```
import pymcpp as mc

DAG = mc.FFGraph()
X, Y = DAG.add_var("X"), DAG.add_var("Y")
F = mc.exp(X*Y) - 2*X**2*Y + 3
SGF = DAG.subgraph([F])

DAG.eval(SGF, [F], [X, Y], [0.5, 1.0])     # [4.1487]

IX, IY = mc.Interval(-1., 1.), mc.Interval(0., 2.)
DAG.eval(SGF, [F], [X, Y], [IX, IY])       # [-0.8647, 10.3891]

xMC, yMC = mc.McCormick(IX, 0.5), mc.McCormick(IY, 1.0)
[FMC] = DAG.eval(SGF, [F], [X, Y], [xMC, yMC])
FMC.cv, FMC.cc                             # 2.0, 8.5756 at (0.5, 1)

CM = mc.SCModel(3)                         # Chebyshev model, order 3
[FCM] = DAG.eval(SGF, [F], [X, Y], [mc.SCVar(CM, 0, IX), mc.SCVar(CM, 1, IY)])
FCM.B                                      # [-1.5230, 7.7258]

SUP = mc.PWLSModel(2)                      # superposition, 8 segments/variable
[FSU] = DAG.eval(SGF, [F], [X, Y],
```

DRAFT

```
                    [mc.PWLSVar(SUP, 0, IX, 8), mc.PWLSVar(SUP, 1, IY, 8)])
FSU.l(), FSU.u()                              # -0.8647, 9.3891
```

A companion notebook reproduces these results, with differentiation, decomposition and other arithmetics; it runs in the browser without installation.[1]

Research impact statement

MC^{++} is the implementation of record for a line of set arithmetics published in peer-reviewed journals: McCormick-based relaxations of algorithms (Mitsos et al., 2009), multivariate McCormick relaxations (Tsoukalas & Mitsos, 2014), McCormick–Taylor models (Bompadre et al., 2013; Sahlodin & Chachuat, 2011), Chebyshev model arithmetic (Rajyaguru et al., 2017), ellipsoidal arithmetic (Villanueva et al., 2015) and relaxation via separable estimators (Zha et al., 2026), each developed and validated in MC^{++} or its predecessor libMC.

Its use extends beyond its authors. MAiNGO (Bongartz et al., 2018) is built on MC^{++} and vendors it as a dependency, and Pyomo (Hart et al., 2011) wraps it for McCormick relaxations. Applications include surrogate-based global optimization with embedded neural networks (Schweidtmann & Mitsos, 2019) and Gaussian processes (Schweidtmann et al., 2021), through MeLOn (https://git.rwth-aachen.de/avt-svt/public/MeLOn), and model-based design of experiments running entirely on MC^{++} DAGs (Sandrin et al., 2025). Despite more than a decade of such use, MC^{++} has never had a software paper, and users have cited method papers in its place.

AI usage disclosure

Anthropic's Claude models (Claude Opus 4.8, Opus 5, Opus 5.5 and Fable 5.1) assisted in debugging and improving parts of the C++ code and Python binders, and in drafting sections of this paper; OpenAI's ChatGPT was used to review drafts. The authors reviewed, edited and validated all AI-assisted output and made the core design decisions. AI-assisted code was accepted only after passing the repository's `check` target, which gates version consistency, the ABI fingerprint, the Python check scripts and a warning-free documentation build. Every listing in this paper was executed against the released `pymcpp` wheel, and the companion notebook reproduces each quoted value.

Acknowledgements

This work was supported by UKRI Engineering and Physical Sciences Research Council grant EP/W003317/1 (ADOPT). We thank the contributors listed in the `AUTHORS` file, in particular Dominik Bongartz, Boris Houska, Alexander Mitsos, Jaromil Najman, Nikola Peric, Jai Rajyaguru, Ali M. Sahlodin, Mario E. Villanueva and Yanlin Zha, for the arithmetic implementations. We are grateful to Terrence Crombie, Tanuj Karia, Radoslav Paulen, Yukun Wang, Hongzhen Zhang and Lifeng Zhang for exercising the library and reporting the defects they found.

References

Althoff, M. (2015). An introduction to CORA 2015. *Proceedings of the Workshop on Applied Verification for Continuous and Hybrid Systems (ARCH14-15)*, *34*, 120–151. https://doi.org/10.29007/zbkv

[1] https://colab.research.google.com/github/omega-icl/mcpp/blob/version-5/notebook/joss_paper.ipynb

DRAFT

Andersson, J. A. E., Gillis, J., Horn, G., Rawlings, J. B., & Diehl, M. (2019). CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, *11*(1), 1–36. https://doi.org/10.1007/s12532-018-0139-4

Bell, B. M. (2026). *CppAD: A package for C++ algorithmic differentiation*. Computational Infrastructure for Operations Research (COIN-OR). https://github.com/coin-or/CppAD

Belotti, P., Lee, J., Liberti, L., Margot, F., & Wächter, A. (2009). Branching and bounds tightening techniques for non-convex MINLP. *Optimization Methods and Software*, *24*(4-5), 597–634. https://doi.org/10.1080/10556780903087124

Benet, L., Hénot, O., Richard, B., & Sanders, D. P. (2026). *IntervalArithmetic.jl*. https://doi.org/10.5281/zenodo.3336308

Benet, L., & Sanders, D. P. (2019). TaylorSeries.jl: Taylor expansions in one and several variables in Julia. *Journal of Open Source Software*, *4*(36), 1043. https://doi.org/10.21105/joss.01043

Benet, L., Sanders, D. P., & contributors. (2026). *TaylorModels.jl: Rigorous Taylor models in Julia*. https://github.com/JuliaIntervals/TaylorModels.jl

Bompadre, A., Mitsos, A., & Chachuat, B. (2013). Convergence analysis of Taylor models and McCormick-Taylor models. *Journal of Global Optimization*, *57*(1), 75–114. https://doi.org/10.1007/s10898-012-9998-9

Bongartz, D., Najman, J., Sass, S., & Mitsos, A. (2018). *MAiNGO – McCormick-based algorithm for mixed-integer nonlinear global optimization*. Process Systems Engineering (AVT.SVT), RWTH Aachen University. http://permalink.avt.rwth-aachen.de/?id=729717

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs*. https://github.com/jax-ml/jax

Brönnimann, H., Melquiond, G., & Pion, S. (2006). The design of the Boost interval arithmetic library. *Theoretical Computer Science*, *351*(1), 111–118. https://doi.org/10.1016/j.tcs.2005.09.062

Bussieck, M. R., & Meeraus, A. (2004). General algebraic modeling system (GAMS). In J. Kallrath (Ed.), *Modeling languages in mathematical optimization* (Vol. 88, pp. 137–157). Springer. https://doi.org/10.1007/978-1-4613-0215-5_8

Bynum, M. L., Hackebeil, G. A., Hart, W. E., Laird, C. D., Nicholson, B. L., Siirola, J. D., Watson, J.-P., & Woodruff, D. L. (2021). *Pyomo – optimization modeling in Python* (Third, Vol. 67). Springer. https://doi.org/10.1007/978-3-030-68928-5

Chabert, G., & Jaulin, L. (2009). Contractor programming. *Artificial Intelligence*, *173*(11), 1079–1100. https://doi.org/10.1016/j.artint.2009.03.002

Chachuat, B., Houska, B., Paulen, R., Perić, N., Rajyaguru, J., & Villanueva, M. E. (2015). Set-theoretic approaches in analysis, estimation and control of nonlinear systems. *IFAC-PapersOnLine*, *48*(8), 981–995. https://doi.org/10.1016/j.ifacol.2015.09.097

Chen, X., Ábrahám, E., & Sankaranarayanan, S. (2013). Flow*: An analyzer for non-linear hybrid systems. *Computer Aided Verification (CAV)*, *8044*, 258–263. https://doi.org/10.1007/978-3-642-39799-8_18

Fourer, R., Gay, D. M., & Kernighan, B. W. (1990). A modeling language for mathematical programming. *Management Science*, *36*(5), 519–554. https://doi.org/10.1287/mnsc.36.5.519

Griewank, A., Juedes, D., & Utke, J. (1996). Algorithm 755: ADOL-C: A package for the automatic differentiation of algorithms written in C/C++. *ACM Transactions on Mathematical Software*, *22*(2), 131–167. https://doi.org/10.1145/229473.229474

DRAFT

Griewank, A., & Walther, A. (2008). *Evaluating derivatives: Principles and techniques of algorithmic differentiation* (2nd ed.). Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898717761

Gurobi Optimization, LLC. (2026). *Gurobi Optimizer Reference Manual*. https://www.gurobi.com

Hart, W. E., Watson, J.-P., & Woodruff, D. L. (2011). Pyomo: Modeling and solving mathematical programs in Python. *Mathematical Programming Computation*, *3*(3), 219–260. https://doi.org/10.1007/s12532-011-0026-8

Hojny, C., Besançon, M., Bestuzheva, K., Borst, S., & others. (2025). *The SCIP optimization suite 10.0*. https://arxiv.org/abs/2511.18580

Jakob, W., Rhinelander, J., & Moldovan, D. (2017). *pybind11 – seamless operability between C++11 and Python*. https://github.com/pybind/pybind11

Knüppel, O. (1994). PROFIL/BIAS – a fast interval library. *Computing*, *53*(3-4), 277–287. https://doi.org/10.1007/BF02307379

Lerch, M., Tischler, G., Wolff von Gudenberg, J., Hofschuster, W., & Krämer, W. (2006). FILIB++, a fast interval library supporting containment computations. *ACM Transactions on Mathematical Software*, *32*(2), 299–324. https://doi.org/10.1145/1141885.1141893

Lubin, M., Dowson, O., Dias Garcia, J., Huchette, J., Legat, B., & Vielma, J. P. (2023). JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, *15*(3), 581–589. https://doi.org/10.1007/s12532-023-00239-3

Makino, K., & Berz, M. (2006). COSY INFINITY version 9. *Nuclear Instruments and Methods in Physics Research Section A*, *558*(1), 346–350. https://doi.org/10.1016/j.nima.2005.11.109

McCormick, G. P. (1976). Computability of global solutions to factorable nonconvex programs: Part I – convex underestimating problems. *Mathematical Programming*, *10*(1), 147–175. https://doi.org/10.1007/BF01580665

Misener, R., & Floudas, C. A. (2014). ANTIGONE: Algorithms for coNTinuous/Integer global optimization of nonlinear equations. *Journal of Global Optimization*, *59*(2-3), 503–526. https://doi.org/10.1007/s10898-014-0166-2

Mitsos, A., Chachuat, B., & Barton, P. I. (2009). McCormick-based relaxations of algorithms. *SIAM Journal on Optimization*, *20*(2), 573–601. https://doi.org/10.1137/080717341

Mönnigmann, M. (2011). Fast calculation of spectral bounds for Hessian matrices on hyperrectangles. *SIAM Journal on Matrix Analysis and Applications*, *32*(4), 1351–1366. https://doi.org/10.1137/10078760X

Moore, R. E., Kearfott, R. B., & Cloud, M. J. (2009). *Introduction to interval analysis*. Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898717716

Nagarajan, H., Lu, M., Wang, S., Bent, R., & Sundar, K. (2019). An adaptive, multivariate partitioning algorithm for global optimization of nonconvex programs. *Journal of Global Optimization*, *74*(4), 639–675. https://doi.org/10.1007/s10898-018-00734-1

Neumaier, A. (2003). Taylor forms—use and limits. *Reliable Computing*, *9*(1), 43–79. https://doi.org/10.1023/A:1023061927787

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 8024–8035. https://proceedings.neurips.cc/paper_

DRAFT

files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

Rajyaguru, J., Villanueva, M. E., Houska, B., & Chachuat, B. (2017). Chebyshev model arithmetic for factorable functions. *Journal of Global Optimization*, *68*(2), 413–438. https://doi.org/10.1007/s10898-016-0474-9

Sahlodin, A. M., & Chachuat, B. (2011). Convex/concave relaxations of parametric ODEs using Taylor models. *Computers & Chemical Engineering*, *35*(5), 844–857. https://doi.org/10.1016/j.compchemeng.2011.01.031

Sandrin, M., Pantelides, C. C., & Chachuat, B. (2025). Methodological and computational framework for model-based design of parallel experiment campaigns under uncertainty. *Journal of Process Control*, *152*, 103465. https://doi.org/10.1016/j.jprocont.2025.103465

Schweidtmann, A. M., Bongartz, D., Grothe, D., Kerkenhoff, T., Lin, X., Najman, J., & Mitsos, A. (2021). Deterministic global optimization with Gaussian processes embedded. *Mathematical Programming Computation*, *13*(3), 553–581. https://doi.org/10.1007/s12532-021-00204-y

Schweidtmann, A. M., & Mitsos, A. (2019). Deterministic global optimization with artificial neural networks embedded. *Journal of Optimization Theory and Applications*, *180*(3), 925–948. https://doi.org/10.1007/s10957-018-1396-0

Smith, E. M. B., & Pantelides, C. C. (1999). A symbolic reformulation/spatial branch-and-bound algorithm for the global optimisation of nonconvex MINLPs. *Computers & Chemical Engineering*, *23*(4–5), 457–478. https://doi.org/10.1016/S0098-1354(98)00286-5

Tawarmalani, M., & Sahinidis, N. V. (2005). A polyhedral branch-and-cut approach to global optimization. *Mathematical Programming*, *103*(2), 225–249. https://doi.org/10.1007/s10107-005-0581-8

Tsoukalas, A., & Mitsos, A. (2014). Multivariate McCormick relaxations. *Journal of Global Optimization*, *59*(2), 633–662. https://doi.org/10.1007/s10898-014-0176-0

Villanueva, M. E., Rajyaguru, J., Houska, B., & Chachuat, B. (2015). Ellipsoidal arithmetic for multivariate systems. *Computer Aided Chemical Engineering*, *37*, 767–772. https://doi.org/10.1016/B978-0-444-63578-5.50123-7

Wilhelm, M. E., & Stuber, M. D. (2022). EAGO.jl: Easy advanced global optimization in Julia. *Optimization Methods and Software*, *37*(2), 425–450. https://doi.org/10.1080/10556788.2020.1786566

Zha, Y., Villanueva, M. E., Houska, B., & Chachuat, B. (2026). Relaxation via separable estimators: Arithmetic and implementation. *Journal of Global Optimization*, *96*(2), 403–440. https://doi.org/10.1007/s10898-026-01648-z

DRAFT